Python Notes — Loops & Loop Control

Topics: for loops | while loops | break & continue

1. What Is a Loop?

A loop tells Python to repeat a block of code multiple times — either for every item in a sequence, or for as long as a condition remains True. Without loops, you would have to write the same code over and over again manually.

> 💡 **Tip:** There are two main loop types in Python: for (used when you know what to iterate over) and while (used when you repeat until a condition changes).

2. The for Loop

A for loop iterates over a sequence — such as a list, string, or range — and runs the indented block once for each item in that sequence.

Syntax

```
for variable in sequence:
    # code to run for each item
```

The `variable` is a temporary name given to each item on each pass. You choose the name; Python assigns its value automatically.

Example 1 — Iterating Over a List

```
numbers = [1, -2, 3, -4, 5, 6, -7, -8, 9, 10]
positive_number_count = 0

for num in numbers:
    if num > 0:
        positive_number_count += 1

print("final count of positive number is: ", positive_number_count)

# Output: final count of positive number is:  6
```

Walkthrough: On each iteration, num takes the next value from the list. If that value is greater than 0, the counter goes up by 1. After all 10 items are visited, the result is printed.

Example 2 — Iterating Over a String

```
input_string = "python"
reversed_str = ""

for char in input_string:
    reversed_str = char + reversed_str

print(reversed_str)

# Output: nohtyp
```

A string is just a sequence of characters. The loop visits each character one by one. Each new character is prepended to the front of reversed_str, building the reversed string step by step.

> 📝 **Note:** On each pass: char='p' → 'p', char='y' → 'yp', char='t' → 'typ', and so on until 'nohtyp'.

Example 3 — Finding a Unique Character

```
input_str = "teeter"

for char in input_str:
    if input_str.count(char) == 1:
        print("The string is: ", char)

# Output: The string is:  r
```

Here, `input_str.count(char)` counts how many times each character appears in the string. Only `'r'` appears exactly once, so it gets printed.

Example 4 — Finding Duplicate Items in a List

```
items = ["apple", "banana", "orange", "apple", "mango"]
unique_item = set()

for item in items:
    if item in unique_item:
        print("duplicate is: ", item)
        break
    unique_item.add(item)

# Output: duplicate is:  apple
```

A `set()` stores only unique values. Each item is checked against the set before being added. If the item is already there, it's a duplicate. The `break` stops the loop immediately (covered in Section 4).

3. The while Loop

A while loop keeps running as long as its condition remains True. Python checks the condition before each iteration; when it becomes False, the loop stops.

Syntax

```
while condition:
    # code to run while condition is True
```

> ⚠️ **Warning:** If the condition never becomes False and there is no break, the loop runs forever. This is called an infinite loop and will freeze your program.

Example — Input Validation Loop

```
while True:
    number = int(input("enter value b/w 1 and 10: "))

    if 1 <= number <= 10:
        print("thanks")
        break
    else:
        print("print valid number try again!!")
```

Here, `while True` creates an intentional infinite loop. The only way out is the `break` statement inside the `if` block, which fires once the user enters a valid number.

This is a very common pattern for input validation: keep asking until you get what you need.

for vs while — When to Use Which

	for loop	while loop
`Use when`	You know what to iterate over (list, string, range)	You repeat until a condition changes
`Stops when`	All items in the sequence are visited	The condition becomes False (or break fires)
`Risk`	Generally safe — finite by nature	Can loop forever if condition never changes
`Example use`	Process each item in a list	Keep asking for input until valid

4. Loop Control: break and continue

Python gives you two keywords to control what happens mid-loop: break exits the loop entirely, and continue skips the rest of the current iteration and moves to the next one.

4.1 break — Exit the Loop Early

```
# Stop as soon as a duplicate is found
items = ["apple", "banana", "orange", "apple", "mango"]
unique_item = set()

for item in items:
    if item in unique_item:
        print("duplicate is: ", item)
        break              # ← exits loop immediately
    unique_item.add(item)

# Output: duplicate is:  apple
```

When `break` runs, Python exits the loop entirely — no more items are checked, even if there are more left in the sequence.

4.2 continue — Skip to the Next Iteration

```
# Print multiplication table of 3, skipping row 5
number = 3

for i in range(1, 11):
    if i == 5:
        continue           # ← skips the print for i=5 only
    print(number, "X", i, "=", number * i)

# Output: 3 X 1 = 3  |  3 X 2 = 6  |  ...  (row 5 is missing)
```

When `continue` runs, Python skips the rest of the block for that iteration and jumps straight back to the top of the loop to check the next value. The loop itself does NOT stop.

break vs continue — Side-by-Side

Keyword	What it does	Loop continues?
`break`	Exits the loop immediately	No — loop ends
`continue`	Skips rest of current iteration only	Yes — next iteration runs

💡 **Tip:** A helpful mental model: break = 'I'm done with this loop entirely.' continue = 'Skip this one, but keep going.'

5. The range() Function

The `range()` function generates a sequence of numbers, commonly used with `for` loops when you need to repeat something a specific number of times.

```
range(stop)               # 0 up to (but not including) stop
range(start, stop)        # start up to (but not including) stop
range(start, stop, step)  # with a custom step
```

Examples

```
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 11):      # 1, 2, 3, 4, ..., 10
    print(i)

for i in range(1, 11, 2):   # 1, 3, 5, 7, 9  (step of 2)
    print(i)
```

⚠️ **Warning:** range(1, 11) goes from 1 to 10 — the stop value (11) is never included. This is a very common off-by-one mistake.

6. Quick Reference

Keyword / Function	Purpose	Works With
`for`	Loop over a sequence item by item	Lists, strings, range, sets, dicts
`while`	Loop while a condition is True	Any boolean condition
`break`	Exit the loop immediately	for and while
`continue`	Skip current iteration, keep looping	for and while
`range()`	Generate a number sequence	Mainly used with for
`set()`	Store unique items, fast lookup	Used with for to track seen items

7. Common Errors to Watch Out For

⚠️ **Error 1:** Infinite while loop — condition never becomes False and no break is used. Always ensure an exit condition exists.

⚠️ **Error 2:** Off-by-one with range() — range(1, 10) only goes up to 9, not 10. Use range(1, 11) to include 10.

⚠️ **Error 3:** Modifying a list while iterating over it — this can cause items to be skipped or repeated unexpectedly. Use a copy or build a new list instead.

⚠️ **Error 4:** Missing indentation inside the loop body → IndentationError. All code that belongs inside the loop must be consistently indented.

⚠️ **Error 5:** Using break or continue outside a loop → SyntaxError. These keywords only work inside for or while blocks.